

T24 / TAFJ Mimicry — Honest Reference

Purpose: Paste this file at the start of a new chat to continue the T24 Engine "how much of real T24/TAFJ can we realistically rebuild" discussion. It captures the full honest assessment, module-by-module percentages, and the concrete engineering patterns for replicating T24's "change behavior without redeploy" capability in any language.

Date compiled: 2026-06-30

0. The Central Question

"Can I build a product that mimics T24 exactly, from A to Z — all modules, services, database structure, and business/financial rules?"

The honest framing that drives every number in this document:

In T24, ~70% of what feels like "behavior" is not code — it is configuration stored as data, interpreted at runtime by TAFJ. When you rebuild in modern code, the *business outcomes* are very mimicable, but the *meta-machine that lets a non-developer change behavior without redeploying* is the part you mostly lose. Every percentage below splits along that line: **business-logic dimension** (high) vs. **config/runtime dimension** (low).

1. Can You Reverse Engineer the TAFJ Runtime?

No — not in any meaningful or legal way.

What TAFJ actually is

TAFJ (Temenos Application Framework Java) is a **proprietary, jBASE-derived bytecode interpreter** that:

- Reads T24 application definitions stored as **data records** (not code files).
- Compiles **BASIC+ source** to its own bytecode at runtime.
- Exposes internal runtime APIs that BASIC+ routines call directly (DB access, transaction control, validation, event hooks).
- Injects per-session context — company code, user profile, override level, business date — into every routine automatically.

Why reverse engineering fails

Barrier	Reality
Legal	TAFJ is closed, commercial IP owned by Temenos. Decompiling/reverse-engineering breaches their license + copyright.
Technical depth	Decompiled JARs = obfuscated bytecode, no field names, no BASIC+ semantics — meaningless without the surrounding runtime context engine.
Data-driven execution	The "logic" lives in T24 <i>records</i> (VERSION, BASIC+ routines stored as rows, APPLICATION definitions). Needs a live licensed T24 data install to be useful.
Scale	~25 years of accumulated runtime quirks and banking edge-cases — not externally documented.
No public interface	No published spec, no SDK, no documented bytecode format.

Verdict

Reverse engineering TAFJ is **legally prohibited, technically futile, and unnecessary**. The right question is *not* "can we clone TAFJ?" but "**what behaviors does TAFJ produce that we need to replicate?**" — and those are buildable legally from scratch.

2. Will an "8-phase build" contain a TAFJ-like black box with identical behavior?

No. A phased microservice build produces a competent, data-driven banking engine *inspired by* T24 patterns. It does **not** contain a TAFJ-equivalent black box and will **not** produce identical behavior.

Why "exactly the same" is impossible

TAFJ is not a feature you checkbox in a late phase — it is the **substrate everything runs on**:

```
Every operation → TAFJ interprets it → using runtime APIs →
  with BASIC+ routines compiled to bytecode →
    reading VERSION / ENQUIRY / APPLICATION records as data →
      inside a per-session context (company, user, override, date)
```

Replacing that substrate with hardcoded microservices is a fundamentally different execution model.

What TAFJ does	What a modern phased build does	Same behavior?
Change logic by editing a data record , no redeploy	Field-level rules only; real logic changes = code + redeploy	No
BASIC+ compiled to bytecode, full runtime API access	Sandboxed JS/Python snippets (fragile, partial)	No
25 years of edge-cases baked into the interpreter	You reimplement the cases you think of	No
One uniform execution context for every module	Each service handles its own context	Approximate

The trap

The danger is the phrase "*a black box like TAFJ.*" Phases 1–6 (the real bank) are mostly achievable. Chasing the last 80% of the runtime gap (the "be TAFJ" phases) is a sinkhole your own analysis already rated unachievable. **Aim for "behaves like a T24-run bank to its users," not "forensic clone of Temenos' runtime."**

3. Full Module-by-Module Mimicry Map (A to Z)

Layer 0 — Runtime & Data Substrate (the foundation)

Component	Mimic %	Why
TAFJ/TAFC runtime VM	15%	Proprietary interpreter + session context engine. Build a dispatch shim, not a VM.
jBASE / hash-file multi-value data model	40%	JSON columns get the <i>shape</i> ; lose native multi-value SQL + FILE.CONTROL machinery.
Data Dictionary (FIELD.DEFINITION, STANDARD.SELECTION — every field is a record)	80%	Metadata table + middleware. Highest-value, low-glory. Build early.
BASIC+ language + local routines	25%	Sandboxed snippets ≠ compiled language with direct runtime-API access.
OFS (Open Financial Service message layer)	90%	Structured message protocol + router. Very mimicable.
VERSION / ENQUIRY engine (no-code screens & reports)	60–65%	Table-driven builder works; deep cross-app joins + runtime hooks do not.

Substrate average $\approx 45\%$ — the ceiling problem: everything above inherits this loss for the *configurability* dimension.

Layer 1 — Core Banking Modules

Module	Business logic %	Config/runtime %	Overall
Customer (CIF)	90%	70%	85%
Account (CASA, balances)	90%	65%	85%
AA — Arrangement Architecture (activity/property classes, conditions, effective dating, simulation)	55%	30%	45% — T24's crown jewel; you get tiers + effective dating, not the full framework or simulation
Loans / Deposits (LD)	75%	50%	70%
Past Due / Delinquency (PD)	70%	45%	65%
Interest & Charges (IC/IM)	80%	50%	70%

Layer 2 — Payments & Transfers

Module	Overall	Why
Funds Transfer (FT)	80%	Lifecycle + GL posting well-understood
Standing Orders (STO) / Direct Debits (DD)	75%	Scheduling + COB-driven execution
Teller (TT)	80%	Denomination handling, cash positions — mechanical
Payments Hub / PW (ISO 20022, routing)	45%	Large configurable orchestration engine; rebuild a subset
SWIFT / Delivery (DE) — MT/MX generation	50%	Can emit MT103/MT202; full DE (network rules, ack/nack, repair) is deep

Layer 3 — Treasury & Capital Markets

Module	Overall	Why
FX (Foreign Exchange)	60%	Deal capture + revaluation doable; full position-keeping harder
Money Market (MM)	55%	
Securities (SC)	35%	Corporate actions, custody, settlement — specialist
Derivatives (DX) / Swaps	25%	Valuation = quant domain, not engineering
Trade Finance — LC / Guarantees / Collections	30%	Document-driven, highly configurable, manual workflows

Layer 4 — Accounting Engine

Component	Overall	Why
General Ledger (double-entry, DR=CR atomic)	85%	Core invariant is clean engineering
Chart of Accounts / CRF / RE. reporting lines*	65%	Configurable category → line → report mapping is the T24-specific part
Multi-currency revaluation & position accounting	60%	Math is fine; the configurable entry-generation engine is deep
Multi-Company / multi-book (CO_CODE)	90%	Pure engineering — add the dimension everywhere from day one

Layer 5 — Batch / COB / Services

Component	Overall	Why
COB (Close of Business) phases	80%	Sequenced batch well-understood

Component	Overall	Why
TSA / Service framework, restartable jobs, DAG	70%	Job registry + topological scheduler; live monitoring + granular restart is the gap
Statements / advices	75%	

Layer 6 — Security, Compliance, Channels

Component	Overall	Why
SPF / USER / maker-checker / override / field-level security	80%	Individually straightforward
Audit (record history, CURR.NO, INPUTTER/AUTHORISER)	90%	Simple, mandatory pattern
AML / Sanctions / FCM	45%	Engine yes; quality needs commercial watchlists + tuned matching
Regulatory reporting (Basel III, IFRS 9, FATCA/CRS, local returns)	10%	Financial-regulation problem, not engineering. Refuse to fake it — needs a specialist + licensed data.
Channels / IRIS / TWS / Browser / Web services	70%	REST + UI gets most user-visible behavior

4. The Weighted Honest Total

Realistic full-system mimic $\approx$ 60–65%.

- **Retail/commercial scope** (Customer, Account, AA-lite, FT, GL, COB, Security): **~75%**
- **True "A to Z"** (incl. Treasury, Trade Finance, full AA, Regulatory): **~45%** — and the last stretch is *years* of specialist work, not engineering sprints.

The brutal one-paragraph truth

You can mimic **what T24 does for a bank's customers and accountants** to **~75%** in a sane timeframe. You **cannot** mimic **what T24 is for a bank's IT department** — a data-driven, no-redeploy, infinitely-configurable runtime — beyond **~25–45%**, because that *is* TAFJ + AA + 25 years of configurability, and that's the part Temenos actually sells. Build the **75% bank**, treat the configurability layer as *inspired-by*, and refuse the regulatory 10% until you have a specialist.

5. "Change Behavior Without Redeploying" — How T24 Does It & How to Replicate It

T24 has **four distinct mechanisms** people lump together as "no redeploy":

- 1. Metadata / Data Dictionary changes (zero code, zero restart)** Every field, table, screen, validation is a *record* (`FIELD.DEFINITION` , `STANDARD.SELECTION` , `VERSION` , `ENQUIRY` , `PGM.FILE`). Add a field / make it mandatory / build a screen → `INPUT` a record; runtime reads it next run. No DDL, no build. **Tools:** the T24 apps themselves, or **Design Studio** (Eclipse IDE over the same records).
- 2. Configuration / rule records (zero code)** Pricing, interest, charges, GL mapping, limits live in parameter tables (`AA.PRODUCT` , `IC.*` , `FT.COMMISSION.TYPE` , `RE.*`). The engine reads the value at runtime.
- 3. Interpreted logic — BASIC+ local routines (code, but special)** Real procedural logic = BASIC+ routines hooked via records (`.API` , `VERSION` validation, `EB.API` , event hooks). Nuance: BASIC+ **is** compiled to TAFJ bytecode, **but** it compiles *one small routine* in seconds and TAFJ **hot-loads** it — no full restart. Feels like no-redeploy.
- 4. The runtime context engine (TAFJ)** Injects per-session context and dispatches every op through the dictionary + hooks. The conductor that makes 1–3 work together.

Honest decomposition: ~70% of "no-redeploy" = mechanisms 1 & 2 (pure data, genuinely no compile). ~30% = mechanism 3 (micro-compile + hot reload that *feels* like no-redeploy).

How to replicate in ANY language

Pattern 1 — Metadata-driven core (replicates 1 & 2, the 70%) — ~85% achievable

```
Don't hardcode tables/fields/rules. Store them as data, read at runtime.
app_definition      (which "applications" exist)
field_definition    (field, type, mandatory?, regex, lookup, default)
version_record      (role → fields visible/mandatory/hidden)
enquiry_definition  (table, filters[], columns[]) → dynamic query builder
rule_table          (pricing, charges, limits, mappings as rows)
```

A generic middleware interpreter reads these before processing any request. Adding a field/validation = `INSERT`, not a deploy. The single highest-value thing to build.

Pattern 2 — Rules-as-data engine (replicates 2) Put rules in tables / a rules engine, not `if` statements. Effective-date rows (`EFFECTIVE_FROM/T0`) so the runtime/COB picks the valid version (T24-style time-travel pricing).

- **Node:** json-rules-engine · **Python:** business-rules / durable_rules · **Java:** Drools · **Any:** JSON-logic / decision tables.

Pattern 3 — Scriptable hooks (replicates 3, the 30%, partially) Store logic *in a table*, load dynamically instead of baking in:

Language	Hot-loadable script mechanism
Node.js	<code>isolated-vm</code> / <code>vm2</code> , or dynamic <code>import()</code> with cache-busting
Python	<code>importlib.reload</code> , <code>RestrictedPython</code> , sandboxed exec service
Java	GraalVM polyglot , Janino, Groovy/JSR-223, <code>URLClassLoader</code> hot-swap — <i>closest to what TAFJ literally does</i>
Go / Rust	embed a VM: Lua (gopher-lua) , Tengo , Wasm , Starlark
Any	Run rules as WebAssembly modules loaded from a table

Caveat: this is the fragile, security-sensitive 30% (sandbox escape, debuggability, perf). Keep it small and audited.

Pattern 4 — The conductor (replicates TAFJ's role) A thin dispatch layer: request → resolve app/version metadata → apply field rules → run pre/post hooks → execute → audit. Your "mini-TAFJ" — a few thousand lines, not a VM.

Verdict on replicating "no-redeploy"

- Mechanisms **1 & 2 (data-driven config & rules): ~85%** in any language. Captures most of what banks mean by "no-redeploy."
- Mechanism **3 (scripted hooks): ~30–40%**, fragile — **Java/GraalVM gets closest** (genuine compile-and-hotload like TAFJ).
- Mechanism **4 (full TAFJ context VM): ~15%** — you build a dispatcher, not a VM.

If the goal is the *behavior* (ops/config staff change rules without a deploy), you can reach it. If the goal is *the mechanism* (a proprietary bytecode VM), you can't and don't need to.

6. Do We Have the TAFJ Binary / `TAFJ.jar` ?

No. No TAFJ binaries, `TAFJ.jar`, jBASE objects, or any Temenos source/bytecode. No Temenos license, repository, or installation. Nothing to give, decompile, or inspect. What's available is **general architectural knowledge** (public-doc / ecosystem level) of *how TAFJ works conceptually* — descriptions of behavior, not the code. Even if a `TAFJ.jar` were obtained, decompiling it would breach Temenos license + copyright and be useless without a licensed T24 data install.

7. Bottom-Line Recommendations

1. **Build the 75% bank** (Core + Payments + GL + COB + Security) — that runs a real bank.
2. **Build the metadata-driven core early** (Pattern 1) — it's the legal, ~85% replication of T24's "no-redeploy" magic and the highest-value investment.
3. **Treat the runtime VM (TAFJ) as inspired-by**, not a clone target. Build a thin dispatcher (Pattern 4), not a virtual machine.
4. **Use scripted hooks sparingly** (Pattern 3) — fragile, security-sensitive; Java/GraalVM if you need it closest.
5. **Refuse regulatory reporting (~10%)** until you have a financial-regulation specialist + licensed data.
6. **Never reverse-engineer TAFJ** — illegal, futile, unnecessary.